Lab 3 - AI-Assisted Refactoring

flyerrr | Branch lab/introduce-repository-seam | October 22, 2026

1. Refactor Rationale

Constraint

The Lovable prototype stored accounts and events in browser localStorage, and the pages called that storage module directly. Login, the home feed, event detail, Tap In, reminders, and create-event imported createUser, authenticateUser, listEventRecords, or createEventRecord from src/data/appDatabase.ts.

That violates the Dependency Inversion Principle. The view depended on one concrete storage engine, so replacing localStorage with an API would mean editing every page. It also mixed two responsibilities in appDatabase.ts: the storage engine (read, write, seed) and the account and event use cases. Session state had already been pulled into src/services/authService.ts. Account records and the event catalog had not.

Solution

I applied Dependency Inversion and Dependency Injection. Use cases now depend on a DatabasePort. Pages depend on UserRepository and EventRepository, which the app injects through RepositoryProvider. localStorage remains the production adapter. An in-memory adapter implements the same port and is used to prove the use cases are not tied to the browser.

Old structure

Pages called functions in src/data/appDatabase.ts, which both defined the use cases and wrote localStorage.

New structure

- src/repositories/contracts.ts - UserRepository and EventRepository
- src/repositories/databasePort.ts - read and save
- src/repositories/localStorageDatabase.ts - production adapter
- src/repositories/inMemoryDatabase.ts - adapter with no browser writes
- src/repositories/userRepository.ts and eventRepository.ts - the use cases, written once
- src/repositories/RepositoryProvider.tsx - injects the bundle into the tree
- src/data/appDatabase.ts - storage engine only (seed, getDatabase, saveDatabase)

Check-in tokens, reminder storage, and the attendance report still import their own stores. Explore still reads the mock event list. Those are the next slices of GitHub issue #15. This change does not add an HTTP client, because the app still has no API.

2. AI Usage Summary

I used Cursor's agent (Composer) on the Lab 3 prompt: audit the Lovable-generated app, refactor one structural problem toward a production boundary, keep the current behavior, and document the prompts, the human edits, and the verification.

Prompt

```
Audit the flyerrr codebase for a structural code smell.
Introduce a repository seam so pages stop calling localStorage
```

```
use cases directly. Keep sign-in, the feed, and event publishing
behaving as they do now. Add tests for the local adapter and
for a second adapter that does not touch localStorage.
```

What Cursor generated

The repository contracts, both storage adapters, the provider, the page call-site updates, and the unit tests. The use-case steps (email normalization, duplicate-account error, role check, event inserted at the front of the list, capacity of 0 stored as 100) were moved as they existed.

Human review changes

- RepositoryProvider memoizes the repository bundle. A new object on every render would retrigger the home feed's refresh effect.

- TypeScript is set to strict: false, and if (!result.ok) did not narrow the result union. tsc reported that result.error did not exist. Login now checks result.ok === false. The message shown to the user is unchanged.

- Sign-in used to read the clock twice, once for the saved user and once for the returned user, so the two timestamps could differ by a millisecond. The repository now stores and returns one lastLoginAt. loginWithUser does not read that field.

- EventDetail returned before two useEffect calls. A missing event id changed the hook count, which breaks the Rules of Hooks. The effects already returned early when the event was missing, so they were moved above the return. The page for a real event is unchanged.

- Corrupt JSON in the storage engine still falls back to a fresh seed. I left that recovery alone. Throwing would wipe a user's local data.

- I did not add an HttpRepository. There is no server. A client that only throws would look like a seam and would not be testable. The in-memory port is the second implementation.

3. Verification

Automated

npx vitest run and npx tsc -p tsconfig.app.json --noEmit. TypeScript completed with no errors. Vitest: 3 files, 12 tests, all passed.

```
src/test/example.test.ts                        1 passed
src/test/authService.test.ts                   6 passed
src/repositories/repositories.test.ts          5 passed
Test Files  3 passed (3)
Tests       12 passed (12)
```

The new tests cover:

- Login, the feed, create-event, event detail, Tap In, and reminders no longer import src/data/appDatabase.

- The seeded student can sign in. The same password with the officer role is rejected.

- Sign-up trims and lowercases the email, persists across a new repository instance, and rejects a duplicate email.

- A published event is first in the catalog, capacity 0 is stored as 100, and the seeded events remain.

- The same sign-up against the in-memory port does not write the flyer_db_v1 key, and that user is invisible to the localStorage adapter.

Manual, on http://localhost:8080

- Student sign-in with student@howard.edu / password123 opens the feed.
- Welcome Week Block Party opens its detail page.
- A full reload stays on the feed. The session survives refresh.
- Profile shows Campus Student.
- Sign out returns to the role picker.
- A wrong password stays on the login page and shows: Invalid email, password, or role.
- Officer sign-in with officer@howard.edu / password123 shows the Create control.
- Publishing Lab Showcase puts that event first on the feed.
- A full reload keeps Lab Showcase at the top.

Protected routes still send a signed-out browser to /login. The existing authService tests still pass, including corrupt session JSON and logout.

Screenshots

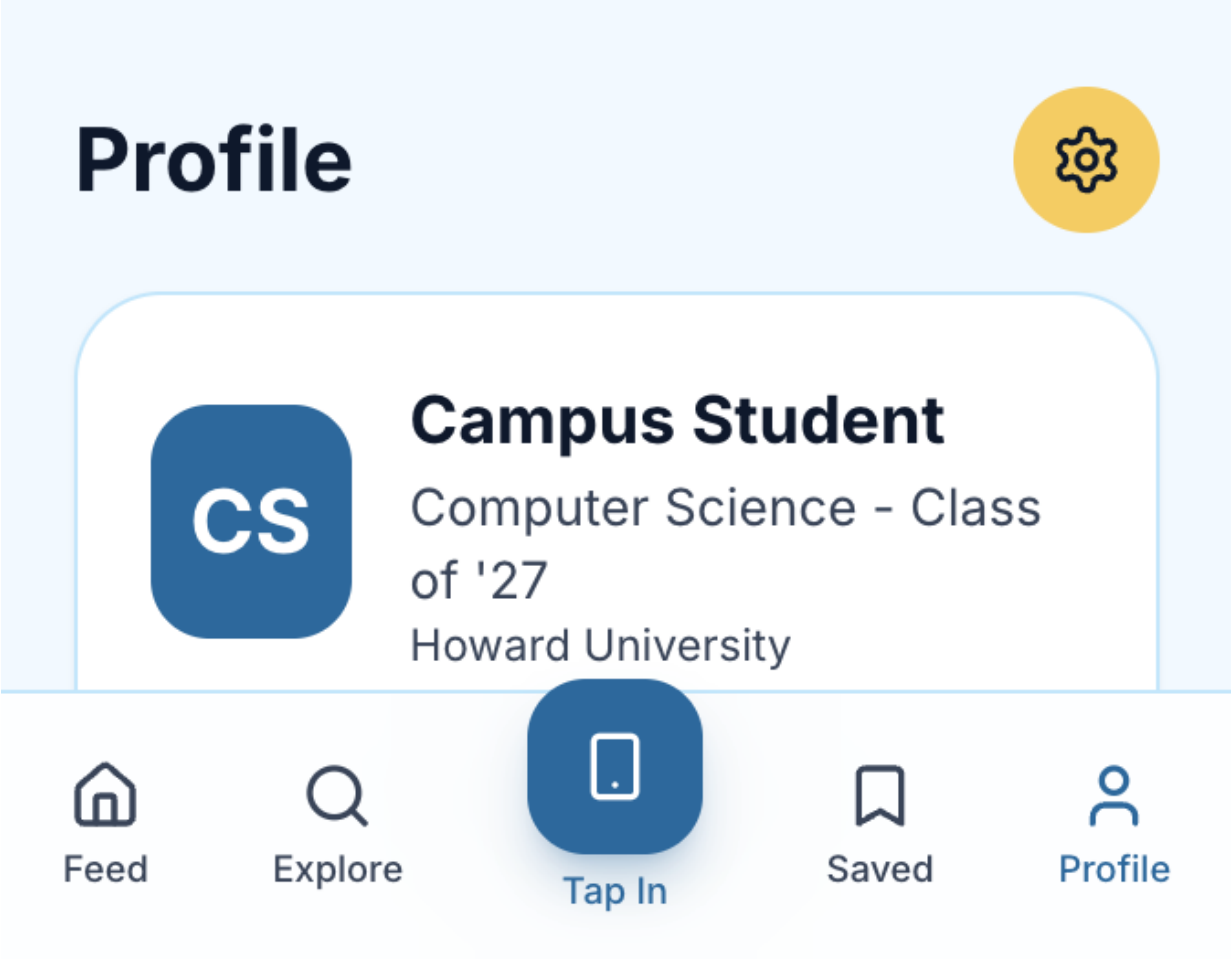


Figure 1. Signed-in student profile after login through UserRepository.

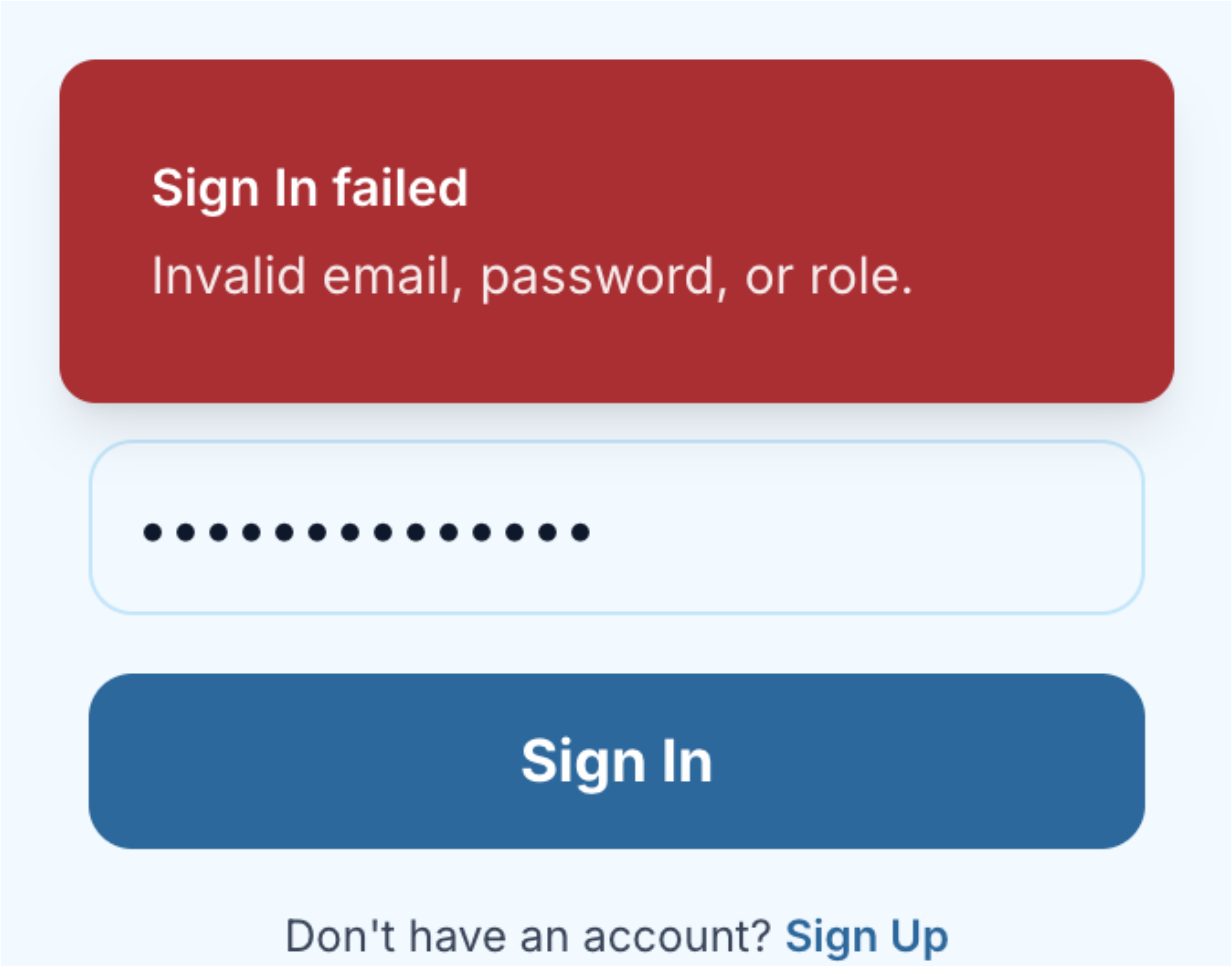


Figure 2. Rejected sign-in. The error string is the same one the storage module used to return.

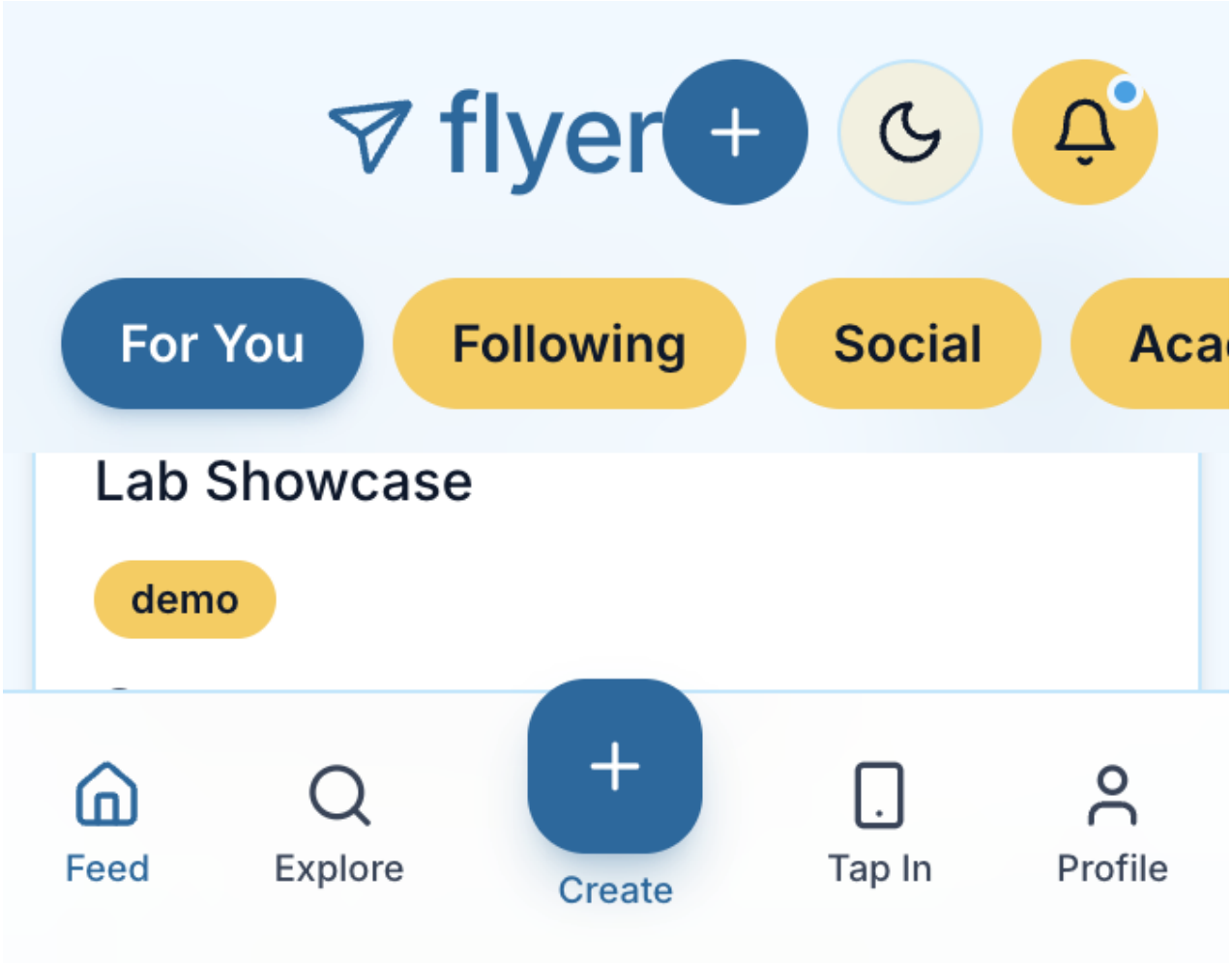


Figure 3. After an officer publishes Lab Showcase, the feed lists it first and the Create control is still the officer action.

What this refactor does not change

Attendance numbers, rewards, and the resume are still seeded or local component state. Check-in tokens are still written to the browser that created them. Explore still searches the mock list, so a newly published event does not appear there. Those were true before this change and are still true.